

WOAC Advanced Powershell Scripting

Site: [CVTE LMS](#)

Course: 170A WOBC/WOIC/WOAC Scripting

Book: WOAC Advanced Powershell Scripting

Printed by: Brandon Larson,

Date: Wednesday, 22 April 2026, 11:00 AM

Table of contents

170A WOAC — Advanced PowerShell Scripting

Chapter 1: PowerShell Scripting Foundations Review

- 1.1 – Objects, Output, and the Pipeline
- 1.2 – Help and Command Discovery
- 1.3 – Variables, Arrays, Hashtables, and Strings
- 1.4 – Control Flow and Looping
- 1.5 – Functions and Parameters
- 1.6 – Returning Structured Output with PSCustomObject
- 1.7 – Error Handling and Script Troubleshooting
- 1.8 – Files, Paths, and Execution Context
- 1.9 – Chapter Review and Practice

Chapter 2: .NET for PowerShell Practitioners

- 2.1 – PowerShell and .NET
- 2.2 – Properties, Methods, and Type Names
- 2.3 – Static vs Instance Members
- 2.4 – Useful .NET Types in Scripts
- 2.5 – Assemblies and Namespaces
- 2.6 – Practical Examples
- 2.7 – Chapter Review and Practice

Chapter 3: Remote Execution, Security, and Operational Constraints

- 3.1 – Debugging Mindset
- 3.2 – WinRM and Remoting Basics
- 3.3 – Authentication, Delegation, and the Double-Hop Problem
- 3.4 – Common Remote Execution Issues
- 3.5 – Language Modes
- 3.6 – PowerShell Abuse and Defensive Context
- 3.7 – DSC, APIs, and Platform Integration
- 3.8 – Chapter Review and Scenario Questions

Chapter 4: Performance and Parallel Execution

- 4.1 – Measuring Performance
- 4.2 – Improving Efficiency Before Parallelizing
- 4.3 – Jobs and Parallel Execution
- 4.4 – Scope, Imports, and Inputs in Jobs
- 4.5 – Parallel Patterns for This Course
- 4.6 – Chapter Review and Practice

Chapter 5: Applied Remote Scripting Workshop

- 5.1 – Remote Query Pattern
- 5.2 – Build Function 1: Remote Scheduled Task Query

- 5.3 – Build Function 2: Remote Event Log Query
- 5.4 – Build Function 3: Deploy a Script File Remotely
- 5.5 – Error Handling and Validation
- 5.6 – Module Assembly
- 5.7 – Orchestrator Pattern
- 5.8 – Parallel Execution from Ops
- 5.9 – Troubleshooting and Cleanup
- 5.10 – Chapter Review and Capstone Readiness

Chapter 6: Capstone

- 6.1 – Capstone Requirements
- 6.2 – Module Standard
- 6.3 – Required Functions
- 6.4 – Orchestrator Standard
- 6.5 – Parallel Execution Standard
- 6.6 – Error Handling Standard
- 6.7 – Validation and Testing Guide
- 6.8 – Submission Checklist

170A WOAC — Advanced PowerShell Scripting

170A WOAC — Advanced PowerShell Scripting

170A WOAC — Advanced PowerShell Scripting

Welcome to the Advanced PowerShell Scripting module of the 170A Warrant Officer Advanced Course. This five-day course builds on your existing PowerShell foundation and prepares you to build, deploy, and orchestrate multi-host automation tools.

 Course Outcome

"In a classroom environment given a Microsoft Windows system with PowerShell, the student will demonstrate advanced PowerShell techniques." — CTSSB Output, 16 February 2024

What you will build

By the end of this course, you will deliver a working capstone solution consisting of:

- A **tools module** (`student_CapstoneTools.psm1`) containing three PowerShell functions
- An **orchestrator script** (`student_Orchestrator.ps1`) that coordinates parallel execution from Ops to four remote targets
- Two **remote query functions** that gather system information from remote hosts
- One **remote deployment function** that installs Sysmon on a remote host

Course environment

System	Role
Ops	Your workstation — all scripts execute from here
System-1	Remote target
System-2	Remote target
System-3	Remote target
Depot	Remote target

How the course is structured

Chapter	Focus	Day(s)
1: Foundations Review	Objects, functions, error handling, execution context	Day 1
2: .NET for PowerShell	Types, properties, methods, static members	Day 1
3: Remote Execution & Security	WinRM, double-hop, language modes, defensive context	Day 2
4: Performance & Parallel	Measure-Command, jobs, scope, parallel patterns	Day 2–3
5: Applied Remote Scripting	Build functions, module assembly, orchestrator	Day 3–4
6: Capstone	Requirements, standards, validation, submission	Day 5

Assessment

Your grade is based on two components:

- **Written exam** — administered end of Day 4, covering knowledge items K1–K11
- **Capstone** — administered Day 5, demonstrating skills S1–S4

Capstone requirements are published on Day 1. You are expected to develop your solution throughout the course in parallel with chapter exercises.

✓ How to use this book

Each chapter contains instructional content, code examples you can run on Ops, common mistake callouts, and a hands-on exercise. The exercises in Chapters 1–4 build foundational skills. The exercises in Chapter 5 teach the patterns you will adapt for the capstone. Chapter 6 defines the capstone standard — no reference implementations are provided.

Chapter 1: PowerShell Scripting Foundations Review

Chapter 1: PowerShell Scripting Foundations Review

Chapter 1: PowerShell Scripting Foundations Review

Chapter Purpose

This chapter refreshes the PowerShell skills you need before the advanced scripting work begins. The goal is not to reteach PowerShell from the beginning. The goal is to make sure you can read, write, test, and troubleshoot the kinds of scripts you will use throughout the course.

This course assumes the following environment:

System	Role in this course
Ops	Student workstation and execution host
System-1	Remote target
System-2	Remote target
System-3	Remote target
Depot	Remote target

Every script you write will execute from Ops. Remote actions will reach into System-1, System-2, System-3, and Depot using PowerShell remoting.

Learning Objectives

- Explain how PowerShell works with objects, properties, and methods. **(K1)**
- Use help and discovery commands to identify command usage and output. **(K1)**
- Apply basic debugging and troubleshooting techniques to identify common script issues. **(K2)**
- Build simple functions with parameters and return structured output. **(K1)**
- Use control flow, path validation, and basic error handling to support reliable script execution. **(K2)**
- Recognize how execution context affects local and remote command behavior. **(K2)**

Chapter Map

Section	Focus
1.1	Objects, output, and the pipeline
1.2	Help and command discovery
1.3	Variables, arrays, hashtables, and strings
1.4	Control flow and looping
1.5	Functions and parameters
1.6	Returning structured output with PSCustomObject
1.7	Error handling and script troubleshooting
1.8	Files, paths, and execution context
1.9	Chapter review and practice

✓ What matters most

By the end of this chapter, you should be comfortable reading command output, inspecting object properties, writing a basic function, returning structured results, and troubleshooting simple script failures. Everything in the capstone depends on these skills.

1.1 – Objects, Output, and the Pipeline

Chapter 1 › 1.1 – Objects, Output, and the Pipeline

1.1 – Objects, Output, and the Pipeline

Most command-line tools return text. PowerShell returns **objects** — structured data with named properties and callable methods. This is the single most important concept in PowerShell, because it means you can filter, sort, select, and export data without parsing strings.

Run this on Ops and look at the output:

```
Get-Process | Select-Object ProcessName, Id, CPU -First 5
```

Each row is not a string — it is a **System.Diagnostics.Process** object. The columns you see are *properties* of that object. PowerShell formats them as a table for display, but underneath, the full object is available.

Inspecting objects with Get-Member

`Get-Member` shows you every property and method available on an object type. This is how you discover what data a command returns and what you can do with it.

```
Get-Service | Get-Member -MemberType Property
```

Pipeline order matters

Filtering and selecting should happen *before* formatting. Once you pipe to `Format-Table` or `Format-List`, the objects are converted to formatting instructions — they are no longer useful data.

```
# Good: filter first, then display
Get-Process | Where-Object CPU -gt 10 | Select-Object ProcessName, CPU

# Bad: format first, then try to filter – this does not work as expected
Get-Process | Format-Table ProcessName, CPU | Where-Object CPU -gt 10
```

⚠ Common mistake — filtering after formatting

If you pipe to `Format-Table` and then try to filter, you are filtering format objects, not process objects. The filter will silently return nothing or behave unpredictably. Always filter **before**

you format.

? Check your understanding

Run both examples above. What does the second one actually return? Why?

1.2 – Help and Command Discovery

Chapter 1 › 1.2 – Help and Command Discovery

1.2 – Help and Command Discovery

You are not expected to memorize every command in PowerShell. You are expected to know how to **find** what you need. The discovery commands are your primary reference throughout this course.

Get-Help

Shows syntax, parameter descriptions, and examples for any command. If help files are not installed, run `Update-Help` once as administrator.

```
Get-Help Get-Service -Examples
```

Get-Command

Finds commands by name pattern, verb, or noun. This is how you answer the question *"is there a command that does X?"*

```
Get-Command -Verb Get -Noun *Service*  
Get-Command -Noun *Process*
```

Get-Alias

Aliases are shortcuts. `dir` is an alias for `Get-ChildItem`. `?` is an alias for `Where-Object`. Aliases are convenient in the console, but **use full command names in scripts** — they are clearer for others and for your future self.

```
Get-Alias dir
```

✓ Course standard

Use full command names in all scripts and exercises. Aliases are acceptable at the interactive console only.

💡 Why this matters

The rest of the course expects you to inspect commands and help files on your own instead

of guessing at syntax. When you encounter an unfamiliar command, your first move should be `Get-Help <command> -Examples` and `<command> | Get-Member` .

1.3 – Variables, Arrays, Hashtables, and Strings

Chapter 1 › 1.3 – Variables, Arrays, Hashtables, and Strings

1.3 – Variables, Arrays, Hashtables, and Strings

Scripting depends on storing and organizing data. You will use variables to hold paths, credentials, target names, and function results throughout this course.

Variables

Variables start with `$` and do not require declaration. PowerShell infers the type from the assigned value.

```
$ComputerName = 'System-1'  
$Count = 42  
$IsReady = $true
```

Arrays

Arrays hold ordered collections. You will use arrays for target lists throughout the course.

```
$Targets = 'System-1', 'System-2', 'System-3', 'Depot'  
$Targets.Count  
$Targets[0]
```

Hashtables

Hashtables store key-value pairs. They are useful when you want to group related configuration values.

```
$Config = @{{  
    BinaryPath = 'C:\\Tools\\Sysmon\\Sysmon64.exe'  
    ConfigPath = 'C:\\Tools\\Sysmon\\sysmonconfig-export.xml'  
}}  
$Config.BinaryPath
```

String expansion

Double-quoted strings expand variables. Single-quoted strings are literal.

```
$Name = 'System-1'  
"Testing $Name"    # expands to: Testing System-1  
'Testing $Name'    # Literal: Testing $Name
```

 Common mistake — wrong quote type

If your string shows a literal `$VariableName` instead of the variable's value, you used single quotes. Switch to double quotes for variable expansion.

1.4 – Control Flow and Looping

Chapter 1 › 1.4 – Control Flow and Looping

1.4 – Control Flow and Looping

Control flow determines what your script does when conditions change. In this course, you will use conditions to validate paths, check connectivity, and decide whether to proceed with remote operations.

If / Else

```
$Path = 'C:\\Tools\\Sysmon\\Sysmon64.exe'

if (Test-Path $Path) {
    'Binary found'
}
else {
    'Binary missing – check the path'
}
```

Foreach

Use `foreach` when you need to process each item in a collection. In this course, that collection is almost always the target list.

```
$Targets = 'System-1','System-2','System-3','Depot'

foreach ($Target in $Targets) {
    "Processing $Target"
}
```

Switch

`switch` is useful when you need to handle multiple known values differently.

```
$Status = 'Failed'

switch ($Status) {
    'Success' { 'All good' }
    'Failed' { 'Investigate this host' }
    default { 'Unknown status' }
}
```

Why this matters

Clear control flow makes your code easier to troubleshoot and extend. In the capstone, your orchestrator will loop through targets and use conditionals to handle per-host results.

1.5 – Functions and Parameters

Chapter 1 › 1.5 – Functions and Parameters

1.5 – Functions and Parameters

Functions let you package logic into reusable tools. Parameters make functions flexible. In this course, every capstone deliverable is a function — so understanding how to build them well is essential.

Basic function structure

```
function Get-Example {  
    param(  
        [string]$ComputerName,  
        [string]$ServiceName  
    )  
  
    Get-Service -Name $ServiceName -ComputerName $ComputerName  
}
```

Once defined, you call it like a cmdlet:

```
Get-Example -ComputerName 'System-1' -ServiceName 'WinRM'
```

Default values

```
function Get-HostInfo {  
    param(  
        [string]$ComputerName = $env:COMPUTERNAME  
    )  
  
    "Querying $ComputerName"  
}  
  
# Both calls work:  
Get-HostInfo                                # uses default (local machine)  
Get-HostInfo -ComputerName 'System-1'      # overrides with System-1
```


Parameter types to know

Type	Use case
[string]	Computer names, paths, service names
[string[]]	Arrays of strings (multiple targets)
[pscredential]	Credential objects from Get-Credential
[switch]	Boolean flags like -Verbose or -Force

✓ Practical standard

A good function should do one clear job, accept input through parameters, and return useful output. Avoid hardcoded target names or paths — those should always be parameters.

⚠ Common mistake — hardcoded values

If you write `$ComputerName = 'System-1'` inside the function body instead of accepting it as a parameter, the function only works for one target. The capstone requires your functions to work against any host passed to them.

1.6 – Returning Structured Output with PSCustomObject

Chapter 1 › 1.6 – Returning Structured Output with PSCustomObject

1.6 – Returning Structured Output with PSCustomObject

Screen text is useful for a human reading the console. Structured output is better for automation, filtering, and export. The standard pattern in this course is to return **predictable objects** from every function.

The PSCustomObject pattern

```
[pscustomobject]@{{  
    ComputerName = 'System-1'  
    Function     = 'Get-RemoteExample'  
    Status       = 'Success'  
    Data         = $SomeResult  
    Error        = $null  
}}
```

This object can be piped to `Format-Table`, `Export-Csv`, `Where-Object`, or any other command. It is real data, not formatted text.

Why consistent properties matter

In the capstone, your orchestrator will collect results from multiple functions across multiple targets. If every function returns the same property names — `ComputerName`, `Status`, `Error` — you can display, filter, and troubleshoot them uniformly.

```
$Results | Where-Object Status -eq 'Failed' | Format-Table ComputerName, Error
```

✓ Course output standard

Every function in this course should return a `[pscustomobject]` with at minimum: **ComputerName**, **Status** (Success or Failed), and **Error** (exception message or `$null`). Additional properties like **Function** and **Data** are recommended.

? Check your understanding

Why is returning an object better than writing directly to the screen with `Write-Host` ? What can you do with an object that you cannot do with screen text?

1.7 – Error Handling and Script Troubleshooting

Chapter 1 › 1.7 – Error Handling and Script Troubleshooting

1.7 – Error Handling and Script Troubleshooting

Errors happen. Good scripts handle them predictably. The difference between a script that crashes and a script that reports failures cleanly is almost always a `try/catch` block and `-ErrorAction Stop`.

The problem with non-terminating errors

Many PowerShell commands produce **non-terminating errors** by default. The command prints red text but *keeps going*. A `catch` block only catches **terminating** errors.

```
# This catch block will NEVER fire – Get-Item writes a non-terminating error
try {
    Get-Item 'C:\\DoesNotExist'
}
catch {
    'Caught it!'    # ← you will never see this
}
```

Common mistake – missing `-ErrorAction Stop`

Without `-ErrorAction Stop`, the command throws a non-terminating error that bypasses your catch block entirely. You see red text on screen, but your error handling code never runs.

The fix: `-ErrorAction Stop`

```
try {
    Get-Item 'C:\\DoesNotExist' -ErrorAction Stop
}
catch {
    $_.Exception.Message
}
finally {
    'Cleanup runs here regardless of success or failure'
}
```

Capturing the error message

Inside a `catch` block, `$_` is the error record. The human-readable message lives at `$_ .Exception.Message`.

```
catch {  
    [pscustomobject]@{  
        ComputerName = $ComputerName  
        Status       = 'Failed'  
        Error        = $_.Exception.Message  
    }  
}
```

Why this matters

The capstone standard expects per-host failures to be captured cleanly instead of crashing the entire script. When your orchestrator processes four remote targets, a failure on System-3 should not prevent results from System-1, System-2, and Depot.

1.8 – Files, Paths, and Execution Context

Chapter 1 › 1.8 – Files, Paths, and Execution Context

1.8 – Files, Paths, and Execution Context

Many script failures come from bad path assumptions or from not understanding *where* a command is actually running. In a multi-host environment like this course, execution context is critical.

Relative vs. absolute paths

```
# Relative – depends on your current directory
Test-Path .\student_CapstoneTools.psm1

# Absolute – works regardless of current directory
Test-Path C:\Users\Student\Desktop\student_CapstoneTools.psm1

# Convert relative to absolute
Resolve-Path .\student_CapstoneTools.psm1
```

Who am I and where am I?

These two questions are the first thing to check when a script behaves unexpectedly.

```
whoami
$env:COMPUTERNAME
Get-Location
```

What changes in a remote session?

When you use `Invoke-Command` to run a script block on `System-1`, the commands inside that block execute *on System-1*. Paths resolve relative to the remote working directory. Variables from your Ops session do not exist inside the remote session.

```
# This runs on Ops
whoami

# This runs on System-1
Invoke-Command -ComputerName 'System-1' -ScriptBlock {
    whoami
    $env:COMPUTERNAME
}
```

Common mistake — local path inside a remote session

If you reference `C:\\Tools\\Sysmon\\Sysmon64.exe` inside an `Invoke-Command` script block, that path resolves *on the remote host*. If the file only exists on Ops, the command will fail.

Check your understanding

If you run `Test-Path .\\myfile.txt` inside an `Invoke-Command` targeting System-1, where does it look for the file?

1.9 – Chapter Review and Practice

Chapter 1 › 1.9 – Chapter Review and Practice

1.9 – Chapter Review and Practice

Before moving on, make sure you can do the following without looking at the chapter material:

- Inspect command output and list useful properties using `Get-Member`
- Use `Get-Help` and `Get-Command` to find commands you have not used before
- Write a function with typed parameters and a default value
- Return a `[pscustomobject]` with consistent properties
- Catch a failure with `try/catch` and capture the exception message
- Validate a path with `Test-Path` before using it
- Explain what changes when a command runs locally vs. inside a remote session

Practice problems

1. Use `Get-Command` to find all commands with the noun `*EventLog*`. Pick one and inspect its output with `Get-Member`.
2. Write a function called `Test-Connectivity` that accepts a `-ComputerName` parameter and returns whether `Test-Connection` succeeds or fails as a `[pscustomobject]`.
3. Demonstrate the difference between single-quoted and double-quoted strings using a variable.
4. Trigger a controlled error using `Get-Item` on a path that does not exist. Capture the error message in a variable and display it.

Chapter 1 Exercise — Build a System Info Reporter

Put together the skills from this chapter into one short script. Work from Ops.

1. Write a function called `Get-SystemSnapshot` that accepts a `-ComputerName` parameter with a default value of `$env:COMPUTERNAME`.
2. Inside the function, use `Get-CimInstance Win32_OperatingSystem` to retrieve the OS caption and last boot time.
3. Return a `[pscustomobject]` with properties: **ComputerName**, **OSCaption**, **LastBoot**, **Status**, and **Error**.
4. Wrap the CIM query in a `try/catch` block. On failure, return a failure object with the exception message.
5. Call your function twice — once with no argument (local), once with an intentionally bad computer name — and pipe both results to `Format-Table`.



Success criteria: Both the success and failure rows display cleanly. The error message is captured in the object, not thrown to the console.

Chapter 2: .NET for PowerShell Practitioners

Chapter 2: .NET for PowerShell Practitioners

Chapter 2: .NET for PowerShell Practitioners

Chapter Purpose

This chapter focuses on the .NET concepts that show up inside PowerShell scripting. The goal is not to turn you into a .NET developer. The goal is to help you recognize the object and type patterns that make PowerShell more powerful.

Learning Objectives

- Explain how .NET types appear in PowerShell output and commands. **(K1)**
- Inspect object properties and methods by using discovery tools. **(K1)**
- Create structured output by using **[PSCustomObject]**. **(K1)**
- Distinguish between static member syntax and instance method usage. **(K1)**
- Apply practical .NET patterns commonly used in PowerShell scripts. **(K1)**

Chapter Map

Section	Focus
2.1	PowerShell and .NET
2.2	Properties, methods, and type names
2.3	Static vs instance members
2.4	Useful .NET types in scripts
2.5	Assemblies and namespaces
2.6	Practical examples
2.7	Chapter review and practice

2.1 – PowerShell and .NET

Chapter 2 › 2.1 – PowerShell and .NET

2.1 – PowerShell and .NET

PowerShell is built on top of the .NET runtime. This is not trivia — it is the reason PowerShell commands can return rich objects instead of plain text. Every object you work with in PowerShell is a .NET object with a type name, properties, and methods.

You can see the .NET version PowerShell is using:

```
$PSVersionTable
```

The **CLRVersion** line tells you which .NET Common Language Runtime is loaded. Every object you see in PowerShell output — processes, services, dates, file info — is backed by a .NET class.

Why this matters

When you pipe `Get-Date` to `Get-Member`, you are looking at the .NET type **System.DateTime** and its members. Understanding this connection helps you find capabilities that are not exposed as standalone cmdlets.

2.2 – Properties, Methods, and Type Names

Chapter 2 › 2.2 – Properties, Methods, and Type Names

2.2 – Properties, Methods, and Type Names

Properties describe data — they tell you what an object *has*. **Methods** perform actions — they tell you what an object can *do*. **Type names** tell you what kind of object you are working with.

Discovering a type name

```
$Date = Get-Date  
$Date.GetType().FullName
```

Listing properties and methods

```
$Date | Get-Member -MemberType Property  
$Date | Get-Member -MemberType Method
```

Accessing properties vs calling methods

Properties use dot notation with no parentheses. Methods use dot notation *with* parentheses.

```
$Date.DayOfWeek          # property – no parentheses  
$Date.AddDays(7)         # method – parentheses required
```

Common mistake — calling a method without parentheses

If you write `$Date.AddDays` without parentheses, PowerShell returns a *description* of the method instead of calling it. Always include parentheses when calling a method.

Check your understanding

When you inspect an object with `Get-Member` and see a member labeled **Property** vs one labeled **Method**, how do you know which syntax to use?

2.3 – Static vs Instance Members

Chapter 2 › 2.3 – Static vs Instance Members

2.3 – Static vs Instance Members

An **instance member** is called on a specific object — you need an object first. A **static member** is called on the *type itself* — no object needed.

Instance members — need an object

```
$Date = Get-Date           # create an instance first
$Date.AddDays(1)           # call method on that instance
$Date.DayOfWeek            # access property on that instance
```

Static members — called on the type with ::

```
[datetime]::Now           # no object needed
[environment]::MachineName # no object needed
[environment]::UserName    # no object needed
```

The double-colon `::` is the static member operator. When you see it in a script, you know the member belongs to the *type*, not to an instance.

? Check your understanding

Which of the examples above require an existing object in a variable, and which do not?

2.4 – Useful .NET Types in Scripts

Chapter 2 › 2.4 – Useful .NET Types in Scripts

2.4 – Useful .NET Types in Scripts

A few .NET types show up frequently in PowerShell scripting. You do not need to memorize the full .NET class library – just know these common ones.

Type	What it gives you	Example
[datetime]	Timestamps, date math	[datetime]::Now
[environment]	Machine name, user, OS info	[environment]::MachineName
[System.IO.Path]	Path manipulation	[IO.Path]::Combine()
[math]	Rounding, max/min	[math]::Round(3.14, 2)
[System.Net.Dns]	DNS lookups	[System.Net.Dns]::GetHostName()

```
# Build a timestamp for logging
[datetime]::Now.ToString('yyyy-MM-dd HH:mm:ss')

# Safe path joining
[IO.Path]::Combine('C:\\Tools', 'Sysmon', 'Sysmon64.exe')

# Get the current machine name
[environment]::MachineName
```

2.5 – Assemblies and Namespaces

Chapter 2 › 2.5 – Assemblies and Namespaces

2.5 – Assemblies and Namespaces

Assemblies package compiled .NET code. Namespaces organize types into groups. In most day-to-day scripting, you will use built-in types without needing to think about this.

```
try {  
    Add-Type -AssemblyName System.Windows.Forms -ErrorAction Stop  
    'Assembly loaded successfully'  
}  
catch {  
    $_.Exception.Message  
}
```

✓ Course scope

Assemblies and namespaces are background knowledge for this course. If you find yourself deep in .NET assembly loading, you have probably overcomplicated the solution.

2.6 – Practical Examples

Chapter 2 › 2.6 – Practical Examples

2.6 – Practical Examples

The best use of .NET in PowerShell scripts is when it makes the code *easier to read* or when PowerShell already exposes the type naturally.

```
[pscustomobject]@{{  
    Timestamp      = [datetime]::Now  
    Workstation    = [environment]::MachineName  
    UserName       = [environment]::UserName  
    SysmonPath     = [IO.Path]::Combine('C:\\Tools', 'Sysmon', 'Sysmon64.exe')  
}}
```

When NOT to use .NET directly

If a cmdlet already does the job, prefer the cmdlet. It is more readable and more discoverable.

```
# Prefer this:  
Test-Path 'C:\\Tools\\Sysmon\\Sysmon64.exe'  
  
# Over this (same result, harder to read):  
[System.IO.File]::Exists('C:\\Tools\\Sysmon\\Sysmon64.exe')
```


2.7 – Chapter Review and Practice

Chapter 2 › 2.7 – Chapter Review and Practice

2.7 – Chapter Review and Practice

Make sure you can do the following:

- Use **GetType().FullName** to identify the .NET type of any object
- Use `Get-Member` to discover properties and methods
- Distinguish between static members (::) and instance members (.)
- Use `[datetime]::Now`, `[environment]::MachineName`, and `[IO.Path]::Combine()` in a script

Practice problems

1. Inspect the type name of `Get-Date` output. Is it a PowerShell type or a .NET type?
2. Create a `[pscustomobject]` that includes a timestamp, username, and temp directory using .NET static members.
3. Find a method on the `[string]` type using `'hello' | Get-Member`. Call it.

Chapter 2 Exercise — .NET Type Explorer

Use .NET discovery skills to inspect and apply types you have not used before.

1. Store the result of `Get-Date` in a variable. Use `Get-Member` to find a method that returns the day of the week. Call it.
2. Use `[System.IO.Path]::GetTempPath()` to find the system temp directory. Combine it with a filename using `[System.IO.Path]::Combine()`.
3. Use `[System.Net.Dns]::GetHostName()` to get the hostname. Compare it to `$env:COMPUTERNAME`.
4. Build a `[pscustomobject]` with: **TimeStamp**, **MachineName**, **TempPath**, and **DayOfWeek**.

Success criteria: You can explain which calls are static members and which are instance methods.

Chapter 3: Remote Execution, Security, and Operational Constraints

Chapter 3: Remote Execution, Security, and Operational Constraints

Chapter 3: Remote Execution, Security, and Operational Constraints

💡 Chapter Purpose

This chapter covers the operational knowledge you need before orchestrating work across remote systems. The focus is on remoting realities, debugging mindset, security context, and common constraints. Several topics — DSC, APIs, Azure — are conceptual rather than hands-on.

Learning Objectives

- Apply an organized approach to debugging PowerShell scripts. **(K2)**
- Explain common remote execution issues, including authentication and delegation concerns. **(K3, K8)**
- Describe the double-hop problem at a practical level. **(K3)**
- Recognize how PowerShell language modes change available behavior. **(K4)**
- Compare common attacker techniques using PowerShell and describe how defenders respond. **(K6)**
- Describe the role of DSC, API interaction patterns, and platform integration at a knowledge level. **(K7, K10, K11)**

Chapter Map

Section	Focus
3.1	Debugging mindset
3.2	WinRM and remoting basics
3.3	Authentication, delegation, and double-hop
3.4	Common remote execution issues
3.5	Language modes
3.6	PowerShell abuse and defensive context

Section	Focus
3.7	DSC, APIs, and platform integration
3.8	Chapter review and scenario questions

3.1 – Debugging Mindset

Chapter 3 › 3.1 – Debugging Mindset

3.1 – Debugging Mindset

Debugging is not about random changes. It is about **controlled observation**. When a script fails, determine what failed, where it failed, and what input triggered it.

The four-step approach

1. **Read the exact error message.** Not the gist — the actual text.
2. **Confirm the execution context.** Where is the command running? As which user?
3. **Test assumptions one at a time.** If you change five things at once, you will not know which fix solved it.
4. **Reduce to the failing command.** Run it alone in the console.

```
# Step 2 – check execution context
```

```
whoami
```

```
$env:COMPUTERNAME
```

```
Get-Location
```

```
# Step 3 – test one assumption
```

```
Test-Path 'C:\\Tools\\Sysmon\\Sysmon64.exe'
```

✓ Practical standard

Do not change five things at once. Verify one assumption at a time.

3.2 – WinRM and Remoting Basics

Chapter 3 › 3.2 – WinRM and Remoting Basics

3.2 – WinRM and Remoting Basics

PowerShell remoting uses **Windows Remote Management (WinRM)** as its transport. In this course, you execute from Ops and remotely interact with System-1, System-2, System-3, and Depot.

Testing connectivity

```
Test-WSMan System-1
```

Three remoting patterns

Pattern	Use case	Command
Interactive session	Manual exploration	Enter-PSSession
One-shot command	Run a single script block	Invoke-Command
Persistent session	Multi-step operations (copy + execute)	New-PSSession + Copy-Item - ToSession

```
# One-shot
Invoke-Command -ComputerName 'System-1' -ScriptBlock { hostname }

# Persistent session
$Session = New-PSSession -ComputerName 'System-1'
Invoke-Command -Session $Session -ScriptBlock { hostname }
Remove-PSSession $Session
```

 Course standard

PSSession remoting is the approved remote execution pattern for the capstone deployment function.

3.3 – Authentication, Delegation, and the Double-Hop Problem

Chapter 3 › 3.3 – Authentication, Delegation, and the Double-Hop Problem

3.3 – Authentication, Delegation, and the Double-Hop Problem

The **double-hop problem** occurs when credentials used for the first remote connection are not automatically delegated to a second resource from inside that remote session.

How it works

Hop 1: Ops → System-1 (succeeds — your credentials are presented)

Hop 2: System-1 → Depot (fails — System-1 does not have your credentials to forward)

The confusing part: the *same command works fine when run directly on System-1* — because there is no second hop when you are already on the machine.

Why you need to know this

In the capstone, your deployment function uses `Copy-Item -ToSession`. This avoids the double-hop because the file transfer is initiated from Ops, not from inside the remote session.

Why this matters

A script may work locally and even remotely, but still fail when a second remote resource is involved. Understanding the authentication path helps you diagnose the real issue.

Check your understanding

In the capstone deployment pattern, which machine initiates the file copy — Ops or the remote target? Why does this matter?

3.4 – Common Remote Execution Issues

Chapter 3 › 3.4 – Common Remote Execution Issues

3.4 – Common Remote Execution Issues

Symptom	Likely cause	First check
WinRM cannot connect	WinRM not enabled or firewall blocking	<code>Test-WSMan <target></code>
Access denied	Wrong credentials or insufficient permissions	<code>Get-Credential</code> – verify username format
Command not found remotely	Module not installed on target	Run <code>Get-Command</code> inside remote session
Path not found	Path exists on Ops but not on target	<code>Invoke-Command { Test-Path ... }</code>
Silently wrong results	Execution context differs	Run <code>whoami</code> inside remote session
Works locally, fails in job	Variable scope – parent vars not inherited	Use <code>-ArgumentList</code>

```
# Quick connectivity and identity check
$Cred = Get-Credential
Invoke-Command -ComputerName 'System-2' -Credential $Cred -ScriptBlock {
    whoami
    hostname
}
```

Common mistake – wrong credential format

If you enter just `student` instead of `DOMAIN\student` in the credential prompt, authentication may fail or authenticate as a different local account.

3.5 – Language Modes

Chapter 3 › 3.5 – Language Modes

3.5 – Language Modes

PowerShell **language modes** control what a session is allowed to do. They are a security mechanism that restricts script capabilities.

Mode	What it allows	When you see it
FullLanguage	Everything — .NET, Add-Type, all cmdlets	Default on most workstations
ConstrainedLanguage	Basic cmdlets and approved types only	AppLocker or WDAC-enforced environments
RestrictedLanguage	Very limited — no script blocks	Rare; specific configurations
NoLanguage	Commands only, no scripting	Extremely locked-down endpoints

Checking the current mode

```
$ExecutionContext.SessionState.LanguageMode
```

Why this matters

If a script behaves differently on one host than another, language mode is one of the first things to check.

Check your understanding

If System-3 is running in ConstrainedLanguage mode, which of the .NET techniques from Chapter 2 would stop working?

3.6 – PowerShell Abuse and Defensive Context

Chapter 3 › 3.6 – PowerShell Abuse and Defensive Context

3.6 – PowerShell Abuse and Defensive Context

PowerShell is widely used for administration and automation. It is also one of the most commonly abused tools by adversaries.

Why attackers use PowerShell

- **Already installed** – no need to drop a binary.
- **Runs in memory** – encoded commands execute without writing files to disk.
- **Deep system access** – WMI, .NET, COM objects, registry, Active Directory.
- **Remote execution** – WinRM and PSRemoting enable lateral movement.

Common attack patterns

Technique	What it looks like
Encoded commands	<code>powershell -EncodedCommand <base64></code>
Download cradles	<code>IEX (New-Object Net.WebClient).DownloadString(...)</code>
Living off the land	Using built-in cmdlets for recon and lateral movement
AMSI bypass	Attempting to disable Antimalware Scan Interface

Defensive controls

Control	What it does
Script Block Logging	Records full content of every script block executed
Transcription	Captures all console input and output
Constrained Language Mode	Restricts .NET access
AMSI	Scans script content at runtime
Code signing	Restricts which scripts can run

 Why this matters

As a 170A, you should understand both the legitimate value of PowerShell and why unusual usage may require investigation. The same remoting and scripting skills you use in the capstone are the same capabilities adversaries exploit.

3.7 – DSC, APIs, and Platform Integration

Chapter 3 › 3.7 – DSC, APIs, and Platform Integration

3.7 – DSC, APIs, and Platform Integration

This section covers three topics at a conceptual level. You will not build scripts around these for the capstone, but you should recognize them for the written assessment.

Desired State Configuration (DSC)

DSC is a PowerShell-driven model for defining how systems *should* be configured, then automatically enforcing that state. You declare the desired state and let the DSC engine handle enforcement.

```
# Conceptual DSC example – not a hands-on exercise
Configuration BaselineConfig {
    Node 'System-1' {
        Service WinRM {
            Name      = 'WinRM'
            State     = 'Running'
            StartupType = 'Automatic'
        }
    }
}
```

API Interactions

PowerShell can interact with web APIs using `Invoke-RestMethod`.

```
$Response = Invoke-RestMethod -Uri 'https://jsonplaceholder.typicode.com/users/1' -Method
Get
$Response.name
$Response.email
```

Azure and Cloud Integration

PowerShell integrates with Microsoft Azure through the `Az` module. Commands like `Connect-AzAccount` and `Get-AzVM` allow you to manage cloud resources.

 Course scope

DSC, API interactions, and Azure integration are conceptual topics. You will encounter them on the written assessment at a recognition level.

3.8 – Chapter Review and Scenario Questions

Chapter 3 › 3.8 – Chapter Review and Scenario Questions

3.8 – Chapter Review and Scenario Questions

Scenario questions

1. Your script works targeting System-1 but fails targeting System-2. Both are in the same domain. What is the **first thing** you should check?
2. A deployment script copies files successfully but the install command fails with "access denied." The same command works interactively on the target. What might explain this?
3. You run `Invoke-Command` to query a file share from inside a remote session, and it fails — but works when you RDP directly. What is this called?
4. A script uses `[datetime]::Now` and works on Ops but throws an error on System-3. What should you check?
5. Your colleague writes a script that downloads and executes a file from the internet using `Invoke-Expression`. Why should a defender be concerned?

Chapter 3 Exercise — Remote Connectivity Validator

Confirm your ability to reach all remote targets before building capstone functions.

1. Write a script that loops through: 'System-1', 'System-2', 'System-3', 'Depot' .
2. For each target, use `Test-WSMan` inside a `try/catch` to check WinRM.
3. For reachable targets, use `Invoke-Command` to run `whoami` and `$ExecutionContext.SessionState.LanguageMode` .
4. Return a `[pscustomobject]` per target with: **ComputerName, WinRMReachable, RemoteUser, LanguageMode, Error.**
5. Display all results with `Format-Table` .

Success criteria: You can confirm connectivity, identify remote context, and handle per-host failures without crashing the loop.

Chapter 4: Performance and Parallel Execution

Chapter 4: Performance and Parallel Execution

Chapter 4: Performance and Parallel Execution

Chapter Purpose

This chapter teaches how to think about efficiency, scale, and concurrency before building the orchestrator. The goal is to understand when parallel execution helps and how to use it safely.

Learning Objectives

- Explain why script performance matters in multi-target operations. **(K5)**
- Measure script runtime and identify avoidable inefficiencies. **(K5)**
- Use PowerShell jobs to run work in parallel. **(K5)**
- Explain how jobs affect scope, imports, and input handling. **(K5)**

Chapter Map

Section	Focus
4.1	Measuring performance
4.2	Improving efficiency before parallelizing
4.3	Jobs and parallel execution
4.4	Scope, imports, and inputs in jobs
4.5	Parallel patterns for this course
4.6	Chapter review and practice

4.1 – Measuring Performance

Chapter 4 › 4.1 – Measuring Performance

4.1 – Measuring Performance

Before you change a script for speed, **measure what it does now**. PowerShell provides `Measure-Command` to time any script block.

```
$Time = Measure-Command {  
    Get-Service | Where-Object Status -eq 'Running'  
}  
  
"Elapsed: $($Time.TotalMilliseconds) ms"
```

For an orchestrator running the same function against 4 targets serially, 2 seconds each means 8 seconds total. Against 50 targets, that is over a minute. Parallel execution reduces wall-clock time by running work concurrently.

✓ Practical standard

Measure before optimizing. The capstone requires parallel execution as a demonstrated skill, but in the real world, use it when it provides real value.

4.2 – Improving Efficiency Before Parallelizing

Chapter 4 › 4.2 – Improving Efficiency Before Parallelizing

4.2 – Improving Efficiency Before Parallelizing

Parallelism is not the first tool to reach for. Start by reducing unnecessary work.

Strategy	Example
Filter early in the pipeline	<code>Get-Service Where-Object Status -eq 'Running'</code>
Avoid repeated lookups	Call <code>Get-Credential</code> once, not once per target
Validate inputs before remote work	Check files exist on Ops before starting sessions
Return only what you need	Use <code>Select-Object</code> to limit properties

```
# Bad: prompts 4 times
foreach ($Target in $Targets) {
    $Cred = Get-Credential
    Invoke-Command -ComputerName $Target -Credential $Cred -ScriptBlock { hostname }
}

# Good: prompts once
$Cred = Get-Credential
foreach ($Target in $Targets) {
    Invoke-Command -ComputerName $Target -Credential $Cred -ScriptBlock { hostname }
}
```

Common mistake – work inside the loop that belongs outside

If you put credential prompts, file validation, or module imports inside a loop, they run repeatedly for no benefit. Move anything that does not depend on the current iteration to *before* the loop.

4.3 – Jobs and Parallel Execution

Chapter 4 › 4.3 – Jobs and Parallel Execution

4.3 – Jobs and Parallel Execution

Jobs let you run work **asynchronously**. Each job runs in its own PowerShell process.

Basic job lifecycle

```
$Job = Start-Job -ScriptBlock { hostname }  
Receive-Job -Job $Job -Wait -AutoRemoveJob
```

Multiple jobs in parallel

```
$Targets = 'System-1','System-2','System-3','Depot'  
  
$Jobs = foreach ($Target in $Targets) {  
    Start-Job -ArgumentList $Target -ScriptBlock {  
        param($ComputerName)  
        Invoke-Command -ComputerName $ComputerName -ScriptBlock { hostname }  
    }  
}  
  
$Results = $Jobs | Receive-Job -Wait -AutoRemoveJob
```

All four jobs start nearly simultaneously. The total wall-clock time is approximately the *slowest* single target, not the sum.

★ CAPSTONE CONNECTION

In the capstone, your orchestrator must dispatch work to four targets in parallel using `Start-Job`.

4.4 – Scope, Imports, and Inputs in Jobs

Chapter 4 › 4.4 – Scope, Imports, and Inputs in Jobs

4.4 – Scope, Imports, and Inputs in Jobs

This is where most students hit their first job-related bug. Jobs run in a **separate process** with their own scope. They do not inherit variables, functions, or module imports from your parent session.

Passing data into a job

```
$Target = 'System-1'
$Cred = Get-Credential

Start-Job -ArgumentList $Target, $Cred -ScriptBlock {
    param($ComputerName, $Credential)
    Invoke-Command -ComputerName $ComputerName -Credential $Credential -ScriptBlock {
        hostname
    }
}
```

Importing a module inside a job

```
Start-Job -ArgumentList $ComputerName, $Credential, $ModulePath -ScriptBlock {
    param($ComputerName, $Credential, $ModulePath)
    Import-Module $ModulePath -Force
    Get-RemoteTaskInfo -ComputerName $ComputerName -Credential $Credential
}
```

⚠ Common mistake – referencing parent variables inside a job

If you write `$Credential` inside a job without passing it through `-ArgumentList`, the variable will be `$null`. The job has no access to your parent session's variables.

? Check your understanding

You import your module on Ops and then start a job. Inside the job, you call a function from the module — but it fails with 'command not found.' Why?

4.5 – Parallel Patterns for This Course

Chapter 4 › 4.5 – Parallel Patterns for This Course

4.5 – Parallel Patterns for This Course

The approved course pattern for parallel execution:

1. Import the tools module on Ops
2. Build the target list
3. Prompt for credentials once
4. Resolve the module path to absolute
5. Start one job per target, passing everything the job needs
6. Import the module inside each job
7. Run the function(s)
8. Receive and review structured output

```
$Targets = 'System-1','System-2','System-3','Depot'
$Credential = Get-Credential
$ModulePath = (Resolve-Path .\course_Tools.psm1).Path

Import-Module $ModulePath -Force

$Jobs = foreach ($Target in $Targets) {
    Start-Job -ArgumentList $Target, $Credential, $ModulePath -ScriptBlock {
        param($ComputerName, $Credential, $ModulePath)
        Import-Module $ModulePath -Force
        Get-RemoteTaskInfo -ComputerName $ComputerName -Credential $Credential
    }
}

$Results = $Jobs | Receive-Job -Wait -AutoRemoveJob
$Results | Format-Table -AutoSize
```

✓ Practical standard

Get the single-target version working first. Then add parallel execution. Debug function logic and job scope issues separately.

4.6 – Chapter Review and Practice

Chapter 4 › 4.6 – Chapter Review and Practice

4.6 – Chapter Review and Practice

Make sure you can answer:

- How do you measure the runtime of a script block?
- Why should you optimize serial performance before adding parallelism?
- What does a job inherit from the parent session? (Answer: nothing.)
- How do you pass variables and credentials into a job?
- Why must you import your module inside each job?

Chapter 4 Exercise – Serial vs. Parallel Comparison

Measure the actual performance difference in your lab environment.

1. Define the target list. Create a script block: `Start-Sleep -Seconds 2; hostname .`
2. **Serial:** Wrap a `foreach` loop in `Measure-Command`, running against each target one at a time.
3. **Parallel:** Wrap a `Start-Job` fan-out in `Measure-Command`, one job per target.
4. Display both elapsed times and calculate the speedup factor.
5. Answer: Why did the parallel version not finish in exactly 2 seconds?

Success criteria: You can demonstrate measurable speedup and explain the sources of overhead.

Chapter 5: Applied Remote Scripting Workshop

Chapter 5: Applied Remote Scripting Workshop

Chapter Purpose

This is the build chapter. You will construct three working functions and an orchestrator. The functions use **different query targets** than the capstone — scheduled tasks, event logs, and script file deployment. The capstone requires you to **adapt these same patterns** to process queries, service queries, and Sysmon deployment.

Learning Objectives

- Build reusable functions that query remote systems and return structured results. **(K8, K9)**
- Use PowerShell remoting to perform remote deployment actions. **(K8)**
- Handle per-host failures without crashing the full workflow. **(K8)**
- Use a simple parallel orchestration pattern from Ops to the four remote targets. **(K8)**
- Recognize how existing function patterns can be repurposed for new tasks. **(K9)**

Chapter Map

Section	Focus
5.1	Remote query pattern
5.2	Build Function 1 — remote scheduled task query
5.3	Build Function 2 — remote event log query
5.4	Build Function 3 — deploy a script file remotely
5.5	Error handling and validation
5.6	Module assembly
5.7	Orchestrator pattern
5.8	Parallel execution from Ops
5.9	Troubleshooting and cleanup
5.10	Chapter review and capstone readiness

 Key principle

The functions you build in this chapter are **not** the capstone functions. They teach the same patterns using different targets. The capstone requires you to apply these patterns to process inventory, service inventory, and Sysmon deployment.

5.1 – Remote Query Pattern

Chapter 5 › 5.1 – Remote Query Pattern

5.1 – Remote Query Pattern

Every remote query function follows the same structure. Learn it once, and you can build any number of query functions by changing what runs inside the script block.

The pattern

1. Accept input by parameter
2. Use `Invoke-Command` with `-ErrorAction Stop`
3. On success: return `[pscustomobject]` with `Status = 'Success'`
4. On failure: return `[pscustomobject]` with `Status = 'Failed'`


```

function Get-RemoteExample {
    param(
        [string]$ComputerName,
        [pscredential]$Credential
    )

    try {
        $Data = Invoke-Command -ComputerName $ComputerName -Credential $Credential -
ScriptBlock {
            # ← your query goes here
            hostname
        } -ErrorAction Stop

        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteExample'
            Status        = 'Success'
            Data           = $Data
            Error          = $null
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteExample'
            Status        = 'Failed'
            Data           = $null
            Error          = $_.Exception.Message
        }
    }
}

```

★ CAPSTONE CONNECTION

The capstone rubric (R1) requires two Get-* functions that accept -ComputerName , return [pscustomobject] , and work against multiple remote hosts. This pattern satisfies all of those requirements.

5.2 – Build Function 1: Remote Scheduled Task Query

Chapter 5 › 5.2 – Build Function 1: Remote Scheduled Task Query

5.2 – Build Function 1: Remote Scheduled Task Query

Your first practice function queries **scheduled tasks** from a remote target. Same pattern from 5.1, different data source.

```
function Get-RemoteTaskInfo {
    param(
        [string]$ComputerName,
        [pscredential]$Credential
    )

    try {
        $Data = Invoke-Command -ComputerName $ComputerName -Credential $Credential -
ScriptBlock {
            Get-ScheduledTask | Where-Object State -eq 'Ready' |
                Select-Object TaskName, State, TaskPath -First 10
        } -ErrorAction Stop

        [pscustomobject]@{{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteTaskInfo'
            Status        = 'Success'
            Data          = $Data
            Error         = $null
        }}
    }
    catch {
        [pscustomobject]@{{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteTaskInfo'
            Status        = 'Failed'
            Data          = $null
            Error         = $_.Exception.Message
        }}
    }
}
```

★ CAPSTONE CONNECTION

For the capstone, you will build `Get-RemoteProcessInventory` using this same pattern. Instead

of `Get-ScheduledTask` , you will use `Get-Process` inside the script block.

5.3 – Build Function 2: Remote Event Log Query

Chapter 5 › 5.3 – Build Function 2: Remote Event Log Query

5.3 – Build Function 2: Remote Event Log Query

Your second practice function queries **recent event log entries**. Same pattern — different data source.

```
function Get-RemoteLogSummary {
    param(
        [string]$ComputerName,
        [pscredential]$Credential
    )

    try {
        $Data = Invoke-Command -ComputerName $ComputerName -Credential $Credential -
ScriptBlock {
            Get-WinEvent -LogName System -MaxEvents 10 |
                Select-Object TimeCreated, Id, LevelDisplayName, Message
        } -ErrorAction Stop

        [pscustomobject]@{{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteLogSummary'
            Status        = 'Success'
            Data          = $Data
            Error         = $null
        }}
    }
    catch {
        [pscustomobject]@{{
            ComputerName = $ComputerName
            Function      = 'Get-RemoteLogSummary'
            Status        = 'Failed'
            Data          = $null
            Error         = $_.Exception.Message
        }}
    }
}
```

Two functions built. Both use the same skeleton. The only difference is the script block content.
The pattern is reusable.

 CAPSTONE CONNECTION

For the capstone, you will build `Get-RemoteServiceInventory` . Instead of `Get-WinEvent` , you will use `Get-Service` .

5.4 – Build Function 3: Deploy a Script File Remotely

Chapter 5 › 5.4 – Build Function 3: Deploy a Script File Remotely

5.4 – Build Function 3: Deploy a Script File Remotely

Deployment functions are more complex: validate, connect, copy, execute, verify, and clean up. This function deploys a **log collection script** — not Sysmon. The capstone requires Sysmon deployment using the same multi-step pattern.

```

function Install-RemoteLogCollector {
    param(
        [string]$ComputerName,
        [pscredential]$Credential,
        [string]$ScriptPath
    )

    try {
        if (-not (Test-Path $ScriptPath)) {
            throw "Source script not found: $ScriptPath"
        }

        $Session = New-PSSession -ComputerName $ComputerName -Credential $Credential -
ErrorAction Stop

        try {
            Copy-Item $ScriptPath -Destination 'C:\\Windows\\Temp\\Collect-Logs.ps1' `
                -ToSession $Session -Force -ErrorAction Stop

            $RunResult = Invoke-Command -Session $Session -ScriptBlock {
                & 'C:\\Windows\\Temp\\Collect-Logs.ps1'
            } -ErrorAction Stop

            [pscustomobject]@{
                ComputerName = $ComputerName
                Function      = 'Install-RemoteLogCollector'
                Status        = 'Success'
                Data          = $RunResult
                Error         = $null
            }
        }
        finally {
            if ($Session) { Remove-PSSession $Session }
        }
    }
    catch {
        [pscustomobject]@{
            ComputerName = $ComputerName
            Function      = 'Install-RemoteLogCollector'
            Status        = 'Failed'
            Data          = $null
            Error         = $_.Exception.Message
        }
    }
}

```

★ CAPSTONE CONNECTION

For the capstone, you will build `Install-SysmonRemote`. The pattern is the same: validate →

session → copy binary + config → execute installer → verify with `Get-Service` → cleanup. You will need two `Copy-Item` calls and the Sysmon installer syntax.

 Common mistake – forgetting to clean up sessions

If your function throws an error after creating a session but before the `finally` block, the session leaks. Always use `try/finally` to guarantee cleanup.

5.5 – Error Handling and Validation

Chapter 5 › 5.5 – Error Handling and Validation

5.5 – Error Handling and Validation

Validate required files and inputs **before** launching remote actions.

```
if (-not (Test-Path $ScriptPath)) {  
    throw "Source script not found: $ScriptPath"  
}
```

Both success and failure paths should return the same object shape.

```
# Success output  
[pscustomobject]@{{ ComputerName=$ComputerName; Status='Success'; Error=$null }}  
  
# Failure output  
[pscustomobject]@{{ ComputerName=$ComputerName; Status='Failed';  
Error=$_.Exception.Message }}
```

✓ Practical standard

If a reviewer cannot tell the difference between your success and failure output shapes by reading your code, your error handling needs work.

5.6 – Module Assembly

Chapter 5 › 5.6 – Module Assembly

5.6 – Module Assembly

A module file (`.psm1`) packages your functions for import and reuse.

```
# course_Tools.psm1

function Get-RemoteTaskInfo { # ... full function body ... }
function Get-RemoteLogSummary { # ... full function body ... }
function Install-RemoteLogCollector { # ... full function body ... }

Export-ModuleMember -Function Get-RemoteTaskInfo, Get-RemoteLogSummary, Install-RemoteLogCollector
```

Verifying the import

```
Import-Module .\course_Tools.psm1 -Force
Get-Command -Module course_Tools
```

★ CAPSTONE CONNECTION

For the capstone, your module will be named `student_CapstoneTools.psm1` and will contain your three capstone functions. The structure is identical.

5.7 – Orchestrator Pattern

Chapter 5 › 5.7 – Orchestrator Pattern

5.7 – Orchestrator Pattern

The orchestrator is a coordinator. It imports the module, builds the target list, prompts for credentials, launches work, and collects results. It should be **readable and short**.

```
$Targets = 'System-1','System-2','System-3','Depot'
$Credential = Get-Credential

Import-Module .\course_Tools.psm1 -Force

# Run against one target first to test
Get-RemoteTaskInfo -ComputerName 'System-1' -Credential $Credential
```

✓ Practical standard

Keep the orchestrator readable. If it is longer than 30–40 lines, you have probably put too much logic in it.

5.8 – Parallel Execution from Ops

Chapter 5 › 5.8 – Parallel Execution from Ops

5.8 – Parallel Execution from Ops

Once functions work one target at a time, the orchestrator can fan out work in parallel.

```
$Targets = 'System-1','System-2','System-3','Depot'
$Credential = Get-Credential
$ModulePath = (Resolve-Path .\course_Tools.psm1).Path

Import-Module $ModulePath -Force

$Jobs = foreach ($Target in $Targets) {
    Start-Job -ArgumentList $Target, $Credential, $ModulePath -ScriptBlock {
        param($ComputerName, $Credential, $ModulePath)
        Import-Module $ModulePath -Force
        Get-RemoteTaskInfo -ComputerName $ComputerName -Credential $Credential
    }
}

$Results = $Jobs | Receive-Job -Wait -AutoRemoveJob
$Results | Format-Table -AutoSize
```

5.9 – Troubleshooting and Cleanup

Chapter 5 › 5.9 – Troubleshooting and Cleanup

5.9 – Troubleshooting and Cleanup

When testing repeatedly, sessions and jobs accumulate. Make cleanup a habit.

`Get-PSSession | Remove-PSSession`
`Get-Job | Remove-Job -Force`

Check	Command
Test credentials separately	<code>Invoke-Command -ComputerName System-1 -Credential \$Cred -ScriptBlock { whoami }</code>
Confirm files exist on Ops	<code>Test-Path \$BinaryPath</code>
Confirm remote host is reachable	<code>Test-WSMan System-1</code>
Read failure text from object	<code>\$Results Where-Object Status -eq 'Failed'</code>
Check orphaned sessions	<code>Get-PSSession</code>
Check orphaned jobs	<code>Get-Job</code>

 **Common mistake – accumulating stale sessions**

If you test 10 times without cleanup, you may have 10 open sessions to each target. Run `Get-PSSession | Remove-PSSession` between test runs.

5.10 – Chapter Review and Capstone Readiness

Chapter 5 › 5.10 – Chapter Review and Capstone Readiness

5.10 – Chapter Review and Capstone Readiness

You are ready for the capstone when you can do the following with confidence:

- Build a remote query function using the pattern from 5.1
- Build a remote deployment function using the pattern from 5.4
- Return structured output from every function
- Handle per-host errors without crashing the workflow
- Assemble functions into a module and verify the import
- Write an orchestrator that coordinates work across all targets
- Execute work in parallel using Start-Job

The capstone requires you to adapt, not copy

Capstone function	Chapter 5 equivalent	What changes
Get-RemoteProcessInventory	Get-RemoteTaskInfo	Script block: Get-Process instead of Get-ScheduledTask
Get-RemoteServiceInventory	Get-RemoteLogSummary	Script block: Get-Service instead of Get-WinEvent
Install-SysmonRemote	Install-RemoteLogCollector	Two copies (binary+config), Sysmon install syntax, service verification

Chapter 5 Exercise — End-to-End Dry Run

Use your Chapter 5 functions to validate the full workflow.

1. Import your `course_Tools.psm1` module. Verify all three functions loaded.
2. Run each function against one target. Confirm structured output.
3. Run all three in parallel against all four targets.
4. Clean up: `Get-PSSession | Remove-PSSession` and `Get-Job | Remove-Job -Force`.

Success criteria: All functions return structured output. Parallel execution works. Per-host failures are captured.

Chapter 6: Capstone

Chapter 6: Capstone

Chapter 6: Capstone

Chapter Purpose

This chapter defines the capstone standard. It tells you what to build, how it will be evaluated, and how to test before submission. **No reference implementations are provided.**

Deliverables

- A tools module (.psm1) containing the required functions
- An orchestrator script (.ps1) that runs from Ops
- Two Get-* functions that query remote systems
- One remote deployment function that installs Sysmon

Chapter Map

Section	Focus
6.1	Capstone requirements
6.2	Module standard
6.3	Required functions
6.4	Orchestrator standard
6.5	Parallel execution standard
6.6	Error handling standard
6.7	Validation and testing guide
6.8	Submission checklist

6.1 – Capstone Requirements

Chapter 6 › 6.1 – Capstone Requirements

6.1 – Capstone Requirements

Your capstone must run from Ops and operate against System-1, System-2, System-3, and Depot. It must include:

- Two remote query functions using the `Get-*` naming pattern
- One remote deployment function for Sysmon
- A module (`.psm1`) containing those functions
- An orchestrator script (`.ps1`) that imports the module and coordinates execution

Assessment standard

This is an individual assessment. Each student submits their own artifacts. Requirements are published now — develop your solution throughout the course.

6.2 – Module Standard

Chapter 6 › 6.2 – Module Standard

6.2 – Module Standard

Your module should be simple, readable, and focused on reusable functions.

- Contains all three required functions
- Uses `Export-ModuleMember` to export them
- Imports cleanly with `Import-Module -Force`
- Passes `Get-Command -Module` verification

6.3 – Required Functions

Chapter 6 › 6.3 – Required Functions

6.3 – Required Functions

The approved minimum set:

- `Get-RemoteProcessInventory` — queries process information from a remote target
- `Get-RemoteServiceInventory` — queries service information from a remote target
- `Install-SysmonRemote` — deploys Sysmon to a remote target

Query function requirements

- Accept `-ComputerName` and `-Credential` parameters
- Use `Invoke-Command` with `-ErrorAction Stop`
- Return `[pscustomobject]` on both success and failure

Deployment function requirements

- Accept `-ComputerName`, `-Credential`, `-BinaryPath`, `-ConfigPath`
- Validate source files before creating a session
- Use `New-PSSession` + `Copy-Item -ToSession`
- Execute the Sysmon installer remotely
- Clean up the session in a `finally` block

Pattern reference

The query pattern is in Ch5 sections 5.1–5.3. The deployment pattern is in 5.4. Adapt those patterns to the capstone targets.

6.4 – Orchestrator Standard

Chapter 6 › 6.4 – Orchestrator Standard

6.4 – Orchestrator Standard

The orchestrator should:

- Run on Ops
- Accept target list and module path as parameters with defaults
- Import the module
- Prompt for credentials
- Launch work against all targets
- Collect and display results

```
param(  
    [string[]]$ComputerName = @( 'System-1','System-2','System-3','Depot' ),  
    [string]$ModulePath = '.\\student_CapstoneTools.psm1'  
)
```

6.5 – Parallel Execution Standard

Chapter 6 › 6.5 – Parallel Execution Standard

6.5 – Parallel Execution Standard

The orchestrator must use parallel execution. Requirements:

- Use `Start-Job` to dispatch one job per target
- Pass computer name, credentials, and module path via `-ArgumentList`
- Import the module inside each job
- Collect results with `Receive-Job -Wait -AutoRemoveJob`

 Common mistake – forgetting to import the module inside the job

Jobs run in a separate process. If you do not call `Import-Module` inside the job, your functions will not be available.

 Testing standard

Verify functions work against a single target before adding parallel execution.

6.6 – Error Handling Standard

Chapter 6 › 6.6 – Error Handling Standard

6.6 – Error Handling Standard

Each function should return structured results for both success and failure. A failure on one target should not stop all other work.

- Every remote operation is wrapped in `try/catch`
- Commands that can fail use `-ErrorAction Stop`
- Success and failure objects have the **same property names**
- The orchestrator completes regardless of individual failures

Practical standard

The capstone should fail cleanly, not fail noisily. A reviewer should be able to look at your output table and immediately identify which targets had problems.

6.7 – Validation and Testing Guide

Chapter 6 › 6.7 – Validation and Testing Guide

6.7 – Validation and Testing Guide

Step-by-step validation

1. Import your module: `Import-Module .\student_CapstoneTools.psm1 -Force`
2. Verify functions loaded: `Get-Command -Module student_CapstoneTools`
3. Test each query function against **one** target
4. Test the deployment function against **one** target
5. Verify: `Invoke-Command -ComputerName <target> -ScriptBlock { Get-Service Sysmon* }`
6. Run the orchestrator against all four targets
7. Test with one intentionally bad target name
8. Clean up sessions and jobs

Common mistake – testing only the happy path

If you only test against targets that work, you will not know whether your error handling is correct until the grader runs it against a failing target.

6.8 – Submission Checklist

Chapter 6 › 6.8 – Submission Checklist

6.8 – Submission Checklist

Item	Verified
Module file (.psm1) present	☐
Orchestrator file (.ps1) present	☐
Two remote query functions present	☐
One remote deployment function present	☐
All functions accept -ComputerName	☐
All functions return [pscustomobject]	☐
Error handling wraps all remote operations	☐
Parallel execution implemented	☐
Module imports inside each job	☐
Tested from Ops to all four targets	☐
Tested with at least one failure scenario	☐
Sessions and jobs cleaned up	☐

? Check your understanding

If you are unsure whether something is required, compare it to the capstone standard in this chapter. Keep the solution clear and dependable.